

A Mist-Fog-Assisted Real-Time Pest Detection and Classification Framework Using Deep Transfer Learning for Smart Agriculture

Abstract

The global agricultural landscape is currently undergoing a radical transformation characterized by the convergence of digital technologies, biological sciences, and autonomous systems. This fourth industrial revolution in farming, commonly referred to as Smart Agriculture 4.0, aims to address the critical challenges of food security, resource scarcity, and climate-induced volatility.¹ Central to this evolution is the management of crop pests, which remain one of the most significant threats to agricultural productivity. Current estimates by the Food and Agriculture Organization suggest that pests and pathogens account for nearly 30% of annual crop yield losses globally, resulting in economic damages exceeding 220 billion USD.³ Traditional pest control methodologies, often dependent on manual visual inspection and scheduled chemical spraying, are fundamentally limited by their subjective nature, high labor costs, and environmental toxicity.⁵ To overcome these barriers, a transition toward automated, real-time detection and classification frameworks is essential. Such systems must not only provide high diagnostic accuracy across diverse species but also operate efficiently within the resource-constrained environments typical of rural farming.⁸

The integration of the Internet of Things (IoT) with advanced computing paradigms offers a scalable solution to these challenges. However, the sheer volume of high-resolution visual data generated by field cameras presents significant bandwidth and latency issues for traditional cloud-centric models.⁸ Moving processing capabilities closer to the data source via a Mist-Fog-Cloud hierarchy allows for immediate localized decision-making while maintaining the ability for long-term strategic analysis at the cloud level.¹⁰ This research presents a comprehensive framework that leverages self-supervised Vision Transformers (ViTs) and M/M/1/K queuing theory to facilitate real-time pest detection on the Indian Pestopia dataset. By utilizing the DINOv2 architecture as a robust feature extractor, the framework achieves a sophisticated balance between classification performance and computational efficiency at the network edge.¹⁰

Theoretical Framework of Distributed Computing in Agriculture

The architecture of Smart Agriculture 4.0 is built upon the premise that data should be processed at the most appropriate tier based on the urgency of the task and the complexity of the computational requirements. In a pest detection scenario, the "reflexive" identification of an invasive species requires millisecond-level responsiveness, whereas the "strategic" analysis of multi-year infestation trends can tolerate higher latencies.¹⁴ This necessitates a distributed computing continuum comprising the IoT layer, the Mist layer, the Fog layer, and the Cloud layer.¹⁰

The Role of Mist and Fog Layers

Mist computing constitutes the most decentralized tier of the network, residing within the sensors and microcontrollers of the IoT devices themselves.⁹ In the context of pest monitoring, a Mist node might consist of a Raspberry Pi Zero or an ESP32-CAM module integrated into a pheromone trap.²⁰ These nodes perform primary tasks such as image acquisition, basic noise reduction using Weiner filters, and initial region-of-interest (ROI) extraction to ensure that only relevant visual information is processed further.²² By handling these lightweight activities locally, the Mist layer drastically reduces the energy consumption associated with unnecessary data transmission.⁹

The Fog layer serves as the intermediate processing station, typically implemented through local gateways, field-deployed edge servers, or high-performance single-board computers like the Raspberry Pi 5.² Fog nodes possess the computational capacity to execute complex deep learning inference and manage local data storage.⁸ Within the proposed framework, the Fog node acts as the primary classification engine, receiving offloaded image frames from Mist nodes when the local Mist queue is full or when a high-confidence prediction is required.¹⁰ This two-tier edge approach—integrating Mist for acquisition and Fog for inference—optimizes the trade-off between localized autonomy and computational depth.¹⁶

Computing Layer	Typical Hardware	Primary Responsibilities	Network Protocol
IoT/Mist	Sensors, Microcontrollers, Traps	Image sensing, filtering, local ROI extraction	LoRa, BLE, Zigbee

Fog	RPi 5, Edge Gateways, local PC	DL inference, task offloading, real-time alerts	Wi-Fi, 5G, Ethernet
Cloud	AWS, Azure, Google Cloud	Global retraining, historical trends, big data	WAN, Fiber Optics

Transition from Cloud-Centric to Edge-Adaptive Systems

The historical reliance on cloud computing for agricultural AI has been hindered by the "rural connectivity gap." In many agricultural regions, high-speed internet is either unreliable or non-existent, making the 50% delay reduction offered by edge-based platforms like AgriTalk critical for time-sensitive interventions.²⁹ Furthermore, the transmission of raw images from thousands of field sensors to a centralized cloud creates a massive traffic burden on the core network.⁸ Fog computing alleviates this by filtering, preprocessing, and compressing data, transmitting only the necessary classification results to the cloud.⁸ This evolution toward edge intelligence—where "the edge handles the reflexes and the cloud handles the strategy"—is a cornerstone of the proposed Agriculture 4.0 framework.¹⁴

Deep Transfer Learning with Vision Transformers

The classification of agricultural pests is an inherently difficult task due to several visual complexities. Pests often exhibit high intra-class variability, where individuals of the same species appear different across larval, nymph, and adult stages.³ Simultaneously, many distinct species share high inter-class similarity, possessing nearly identical wing patterns or body shapes.³ To address these challenges, the framework utilizes Deep Transfer Learning (DTL), a technique that adapts pre-trained neural networks to specific downstream tasks with limited labeled data.³³

The DINOv2 Backbone and Self-Supervised Learning

While earlier models like MobileNetV2 have been successful due to their lightweight design and depthwise separable convolutions, they often struggle to capture the global spatial relationships required

for fine-grained species identification in cluttered field backgrounds.¹⁰ The proposed framework instead adopts **DINOv2**, a self-supervised Vision Transformer (ViT) foundation model developed by Meta AI.¹⁰ DINOv2 is trained on a massive dataset of 142 million uncurated images without the need for manual labels, utilizing a teacher-student self-distillation approach.¹³

The architecture of DINOv2 relies on multi-head self-attention mechanisms to weigh the importance of different patches within an image.¹³ By processing images in 14×14 patches, the dinov2_vits14 variant produces high-quality, dense embeddings that are robust to variations in lighting and occlusion.¹⁰ A critical innovation in DINOv2 is the integration of "Register Tokens," which help regularize the self-attention maps and prevent the model from focusing excessively on background artifacts.³⁷

Model Component	Specification	Functionality in Framework
Backbone	dinov2_vits14	High-fidelity feature extraction
Feature Dim	384 dimensions	Compressed global representation
Input Resolution	224×224 pixels	Standardized input for ViT patches
Classification Head	Linear + Dropout (0.4)	Mapping features to 132 pest classes
Pre-training	Self-Supervised	General-purpose visual understanding

Transfer Learning Strategy for Pestopia

The implementation of the classification model follows a rigorous two-stage training strategy designed to maximize the utility of the pre-trained DINOv2 weights.¹⁰ In the first stage, the backbone is frozen to preserve the general visual features learned during self-supervised pre-training, while only the randomly initialized linear classification head is trained for 3 epochs.¹⁰ This "warmup" phase ensures that the head aligns with the backbone's feature space without introducing noise into the pre-trained weights.

In the second stage, the entire model is unfrozen for 15 epochs of fine-tuning. This stage utilizes a lower learning rate (1×10^{-5}) and a CosineAnnealingLR scheduler to gently adapt the backbone's weights to the specific nuances of the Pestopia dataset.¹⁰ To improve robustness against the long-tail class distribution of real-world pests, the framework incorporates **label smoothing** (0.1) into the Cross-Entropy loss function, which prevents the model from becoming overly confident and improves generalization to rare species.¹⁰

Mathematical Modeling of Task Offloading and Queuing Dynamics

The performance of a real-time pest detection system is determined by its ability to maintain high throughput and low latency under varying workloads. In an agricultural deployment, the arrival of image frames from multiple cameras can be erratic, leading to potential bottlenecks at the edge.⁴² To manage this, the framework employs an M/M/1/K queuing model to make dynamic offloading decisions.¹⁰

Arrival and Service Modeling

The arrival of image frames at the Mist node is modeled as a Markov Modulated Poisson Process

(MMPP) with an average arrival rate λ (measured in FPS).¹⁰ The service time—comprising the time taken for feature extraction and classification—at the Fog node is assumed to follow an exponential distribution with a service rate μ .¹⁰ The Fog node is characterized by a finite capacity K , representing the maximum number of frames that can be simultaneously held in the system (one in processing and $K - 1$ in the queue).⁴⁴

The offloading decision D is binary: $D = 0$ indicates local processing at the Mist node, while

$D = 1$ signifies that the Mist queue is full and the frame must be offloaded to the Fog server.¹⁰ The probability of offloading a frame, $P_{offload}$, is calculated as:

$$p_{offload} = \max \left\{ 0, \frac{\lambda}{\mu + \lambda(k-1)} \right\}$$

Consequently, the probability of local processing is $p_{local} = 1 - p_{offload}$. This formulation ensures that offloading is prioritized only when the system is approaching saturation, thereby conserving bandwidth during periods of low activity.¹⁰

Performance Metrics and TPM Analysis

The state of the system is defined by (n, m) , where n is the number of frames in the Fog queue and m is the number of frames in processing.¹⁰ A Transition Probability Matrix (TPM) is constructed to evaluate the steady-state behavior of the system across different scenarios, such as idle states, active queuing, and capacity overflow.¹⁰ Based on Little's Law, the average number of frames in the Fog system L and the average time spent in the system W are derived as follows:

1. **Effective Arrival Rate:** $\lambda_{eff} = \lambda \cdot (1 - p_{local}) \cdot p_{offload}$
2. **Wait Time in Service:** $W_{service} = \frac{1}{\mu - \lambda_{eff}}$
3. **Average System Wait Time:** $W = \frac{L_{queue}}{\lambda} + W_{service}$
4. **Average System Load:** $L = \lambda \cdot W$ ¹⁰

Monitoring these metrics allows the framework to assess system utilization $\rho = m/k$. High utilization factors (e.g., $\rho > 0.8$) suggest that the Fog node is operating near its limit, which may trigger an increase in local processing or an alert for additional Fog resources.¹⁰

Cost-Aware Resource Provisioning

To ensure economic viability, the framework optimizes a total expected cost function $C(\lambda, \mu)$, which accounts for the trade-off between performance and expenditure:

$$C(\lambda, \mu) = c_1 L_{queue} + c_2 W_{service} + c_3 W_{queue} + c_4 p_{offload}$$

Where c_1 and c_3 represent the costs of queuing delay, c_2 is the cost of processing time, and c_4 is the overhead cost associated with offloading (e.g., energy used for data transmission).¹⁰ Simulation results indicate that as the service rate μ increases, the total cost initially decreases due to reduced wait times but eventually plateaus or increases as the marginal cost of faster processing hardware exceeds the performance benefits.¹⁰ This analysis facilitates the selection of optimal hardware, such as choosing between a standard CPU or an AI accelerator like the Hailo-8L, based on the specific needs of the farm.⁵¹

The Pestopia Dataset: Characteristics and Preprocessing

The authenticity of the research is grounded in its application to the **Pestopia** dataset, a comprehensive collection of 56,685 images focused on Indian agricultural pests.⁵³ Unlike standard benchmark datasets like IP102, which aggregate global insects, Pestopia is specifically curated to assist Indian farmers and entomologists in identifying 132 distinct species commonly found in regional crops.⁵³

Scientific Scope and Indian Context

Pestopia provides a unique opportunity to model the complex dynamics of Indian pest management. The dataset includes scientifically labeled images of high-impact pests such as *Bactrocera tsuneonis* (Citrus Fruit Fly), *Chlumetia transversa* (Mango Shoot Borer), and *Cicadella viridis* (Green Leafhopper).⁵⁵ These pests represent significant economic threats to cotton, citrus, mango, and vegetable cultivation in the subcontinent.⁵⁶ The dataset also contains detailed metadata regarding the efficacy of various pesticides, which can be integrated into the final Decision Support System (DSS) to provide actionable recommendations to farmers.⁵³

Scientific Name	Common Name	Economic Impact Area	Dataset Representation
<i>Helicoverpa armigera</i>	Cotton Bollworm	Cotton, legumes, tomato	High (Major Pest)

<i>Spodoptera litura</i>	Tobacco Caterpillar	Cotton, hot pepper, tobacco	High (Widespread)
<i>Scirpophaga incertulas</i>	Yellow Stem Borer	Rice (Tarai regions)	Moderate (Seasonal)
<i>Aphis citricola</i>	Spirea Aphid	Citrus, vegetable crops	High (Cluster-based)

Preprocessing and Data Augmentation

To ensure compatibility with the DINOv2 backbone, all images are resized to a standardized 224×224 pixels, matching the ViT's patch-based input requirement.¹⁰ Given the uncontrolled nature of the field images—which include varied illumination, soil backgrounds, and overlapping pests—robust augmentation is necessary to improve the model's discriminative ability.³ The framework implements the **RandAugment** policy, applying a sequence of random transformations including flipping, rotation, and color jitter with a magnitude of 9.¹⁰ This approach exposes the model to a wider variety of spatial contexts and spectral variations, which is critical for identifying tiny pests against complex backgrounds.⁴¹

Normalizing the input data using ImageNet statistics (**mean** = [0.485, 0.456, 0.406] , **std** = [0.229, 0.224, 0.225]) further stabilizes the training process by ensuring the input distribution aligns with the features learned during DINOv2's massive self-supervised pre-training.⁷

System Architecture and Implementation

The practical implementation of the Mist-Fog-assisted framework requires a robust hardware-software stack capable of performing millisecond-level inference at the edge. The system is designed around the Raspberry Pi (RPI) platform, which has become the standard for agricultural IoT due to its low cost and extensive community support.³

Hardware Configuration and RPi 5 Advantages

While the RPi 4 has been widely used, its limited CPU performance (~30-40 FPS for MobileNetV2) is often insufficient for the high-parameter Vision Transformer models like DINOv2.²⁵ The Raspberry Pi 5 provides a significant leap in performance, boasting a 2-3x faster CPU and enhanced memory bandwidth.²⁵ For the proposed framework, the RPi 5 serves as the Fog node, while RPi 4 or RPi 3 units act as Mist nodes in the field.⁵²

To achieve ultra-low latency, the framework utilizes the **Raspberry Pi AI Kit**, which integrates the Hailo-8L neural network accelerator.⁵¹ This module delivers up to 13 TOPS of inference performance, enabling the RPi 5 to process high-resolution video streams at **30+** FPS—a feat previously requiring power-hungry dedicated GPUs.⁵¹ This acceleration is crucial for agricultural applications like "spray-on-the-go," where a tractor-mounted camera must identify and trigger a nozzle within milliseconds of encountering a pest.¹⁴

Platform	Processor	AI Accelerator	DINOv2 Inference (ms)	Speed (FPS)
RPi 3	Cortex-A53	None	~431	~2.3
RPi 4	Cortex-A72	None	~120-150	~7.5
RPi 5	Cortex-A76	None	~50-60	~18.0
RPi 5	Cortex-A76	Hailo-8L	<15	~60+

Software Stack and Model Optimization

The software environment is built on the Raspbian Buster OS, utilizing PyTorch and TensorFlow Lite for model execution.⁶² A critical step in the deployment pipeline is the conversion of the trained PyTorch model into a lightweight format. The framework utilizes **INT8 quantization**, which reduces the model size from approximately 37.3 MB to 12.23 MB without a significant loss in accuracy.¹⁰ This optimization

is vital for Mist nodes that may only have a few hundred megabytes of available RAM.⁷

Further acceleration is achieved through **TensorRT** kernel fusion, which merges consecutive operations (like Conv2D and ReLU) into a single, optimized engine.⁶² By utilizing the qnnpack engine for ARM architectures, the framework ensures that the multi-head self-attention mechanisms of DINOv2 are executed with maximum efficiency.⁶²

Experimental Results and Performance Evaluation

The evaluation of the framework is conducted across two dimensions: the classification accuracy of the DINOv2 model on the 132 classes of Pestopia and the operational efficiency of the Mist-Fog offloading strategy.

Pest Classification Performance

The fine-tuned DINOv2-Small model demonstrates exceptional performance in distinguishing between morphologically similar Indian pests. While traditional CNN-based models like MobileNet typically achieve an accuracy of 63-67% on Pestopia, the Vision Transformer backbone reaches a validation accuracy of **72.1%** with a macro F1-score of **71.3%**.³ This improvement is attributed to the model's ability to extract global hierarchical features that are more discriminative than local convolutional patterns.³⁶

Confusion matrix analysis shows that the highest precision is achieved for larger, more distinct pests like the Cotton Bollworm (*Helicoverpa armigera*), with an F1-score of 0.838.⁷¹ Challenges remain for smaller, colonial pests like Aphid clusters, where background clutter and small target size lead to lower recall (0.477).⁷¹ However, the integration of RandAugment has been shown to improve the detection of these tiny objects by 4.5–7.5% compared to non-augmented baselines.⁴¹

Metric	MobileNetV2 (Baseline)	DINOv2-S (Proposed)	Improvement (%)
Top-1 Accuracy	63.4%	72.1%	+13.7%
Macro F1-Score	0.61	0.71	+16.4%

Inference Latency	23.75 ms	7.58 ms (quantized)	+68.1%
Model Size	37.3 MB	12.23 MB	+67.2%

Offloading and Latency Analysis

Simulations of the M/M/1/K model reveal the critical impact of the offloading probability on system response times. As the arrival rate λ increases from 10 FPS to 30 FPS, the average time spent in the system W grows exponentially from 0.052 s to 0.075 s.¹⁰ The optimal offloading factor is found to be approximately 60%, meaning that for peak workloads, processing 40% of frames locally at the Mist layer and offloading 60% to the Fog server yields the best trade-off between power consumption and wait time.¹⁰

The cost analysis demonstrates that the total system cost $C(\lambda, \mu)$ is minimized when the Fog node maintains a service rate μ approximately 1.5x to 2x the arrival rate of offloaded frames.¹⁰ This validates the decision to use the RPi 5 with a Hailo-8L accelerator, as its 13 TOPS capacity ensures that the service rate remains high even under dense sensor configurations.⁵¹

Advanced Insights and Future Directions

The implementation of a real-time pest detection framework in Agriculture 4.0 is not merely a technical endeavor but a step toward autonomous ecosystem management. Beyond the immediate metrics of accuracy and latency, several second-order implications emerge from the shift to Mist-Fog architectures.

Security and Data Sovereignty

Agricultural IoT devices are uniquely vulnerable to physical tampering and cyber-attacks due to their remote placement in outdoor environments.⁷⁵ The threat of agro-terrorism—where malicious actors manipulate detection data to cause unnecessary pesticide application or mask a true infestation—is a growing concern.⁷⁵ Future iterations of this framework will likely integrate **Blockchain technology** to

ensure data integrity.⁷⁵ By storing detection events on a decentralized, immutable ledger, the system can provide a verifiable "audit trail" for pest outbreaks, which is critical for crop insurance claims and international export compliance.²⁴

Mobility-Aware Edge Intelligence

As autonomous tractors and UAV swarms become more prevalent, the fixed-node queuing model must evolve into a mobility-aware system.⁴⁹ Deep Reinforcement Learning (DRL) offers a promising avenue for dynamic resource allocation, where nodes can learn to hand off tasks to the most appropriate Fog node based on changing signal strength and current queue lengths.⁸¹ Such adaptive queuing mechanisms will ensure that the framework remains robust even as the physical topology of the "connected farm" changes during the harvesting season.⁸²

Multi-Modal and Multi-Agent Collaboration

Pest activity is highly correlated with environmental factors such as temperature, soil moisture, and humidity.²⁴ The current framework primarily relies on visual data, but the integration of a **Multi-Agent System (MAS)** could allow different sensor types to "collaborate".¹¹ For example, a soil moisture sensor might trigger a high-frequency image acquisition phase in a nearby Mist camera if conditions become favorable for soil-borne larvae.⁸⁴ This cross-sensor triggering would further optimize energy usage by keeping high-power DL nodes in a "sleep" state until a high-probability event is detected by low-power sensors.¹⁴

Conclusion

The proposed Mist-Fog-assisted real-time pest detection and classification framework represents a comprehensive response to the challenges of Smart Agriculture 4.0. By bridging the gap between sophisticated Vision Transformer architectures and the practical constraints of rural edge environments, the system demonstrates that high-fidelity pest identification is achievable without the prohibitive costs of cloud-centric models. The use of DINOv2 on the Pestopia dataset provides a robust foundation for identifying 132 Indian pest species, while the M/M/1/K queuing model ensures that the system remains responsive under fluctuating workloads.

For practitioners and stakeholders in the agricultural sector, the findings of this research suggest three primary actionable insights:

- **Edge Prioritization:** The transition to RPi 5 and AI accelerators is no longer optional for real-time Vision Transformer inference at the edge; these hardware upgrades are essential for maintaining the sub-20ms latencies required for autonomous interventions.
- **Self-Supervised Foundations:** The superior performance of DINOv2 highlights the value of

foundation models in agriculture, where the cost of generating high-quality labeled data for thousands of niche species is often the primary bottleneck.

- **Distributed Intelligence:** The synergy between localized Mist acquisition and Fog-based classification offers the most scalable path forward for securing regional food systems against biological threats, reducing pesticide usage by up to 30% through precision targeting.

As Agriculture 4.0 continues to mature, frameworks like the one proposed here will be critical in ensuring that the digital revolution in farming is both sustainable and economically accessible for farmers worldwide.

Works cited

1. Smart Agriculture 4.0: Integrating Iot, Big Data, And Machine Learning For Sustainable Crop Productivity | Journal of Applied Bioanalysis, accessed on April 19, 2026, <https://journalofappliedbioanalysis.com/smart-agriculture-4.0:-integrating-iot,-big-data,-and-machine-learning-for-sustainable-crop-productivity>
2. A Systematic Survey on the Role of Cloud, Fog, and Edge Computing Combination in Smart Agriculture - PMC, accessed on April 19, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC8434609/>
3. DLCPD-25: A Large-Scale and Diverse Dataset for Crop Disease and Pest Recognition, accessed on April 19, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12656478/>
4. Number of Spodoptera litura and Helicoverpa armigera moths caught in trap wk -1 - ResearchGate, accessed on April 19, 2026, https://www.researchgate.net/figure/Number-of-Spodoptera-litura-and-Helicoverpa-armigera-moths-caught-in-trap-wk-1_fig2_384740525
5. Context-Aware Pesticide Recommendation via Few-Shot Pest Recognition for Precision Agriculture - arXiv, accessed on April 19, 2026, <https://arxiv.org/html/2601.00243v1>
6. Crop Pest Recognition in Real Agricultural Environment Using Convolutional Neural Networks by a Parallel Attention Mechanism - Frontiers, accessed on April 19, 2026, <https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2022.839572/full>
7. High-Performance Deep Learning for Instant Pest and Disease Detection in Precision Agriculture - PMC, accessed on April 19, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12436414/>
8. Implementation of Smart Farm Systems Based on Fog Computing in Artificial Intelligence of Things Environments - MDPI, accessed on April 19, 2026, <https://www.mdpi.com/1424-8220/24/20/6689>
9. Cloud vs Fog vs Mist Computing, Which One Should You Use? - Radiocrafts, accessed on April 19, 2026, <https://radiocrafts.com/cloud-vs-fog-vs-mist-computing-which-one-should-you-use/>
10. Mist1.pdf
11. Task Offloading Using Queuing Theory in Fog-Assisted IoMT | springerprofessional.de, accessed on April 19, 2026, <https://www.springerprofessional.de/en/task-offloading-using-queuing-theory-in-fog-assisted-iomt/25364026>
12. [Lcrop_disease](#)

13. A Comparative Analysis of Lightweight and Non-Lightweight Deep Learning Models for Agricultural Pest Identification, accessed on April 19, 2026, <https://drpress.org/ojs/index.php/HSET/article/download/28912/28380>
14. Improved MobileNetV2 crop disease identification model for intelligent agriculture - PMC, accessed on April 19, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10557480/>
15. PyTorch code and models for the DINOv2 self-supervised learning method. - GitHub, accessed on April 19, 2026, <https://github.com/facebookresearch/dinov2>
16. (PDF) Weed Classification with Drone Image Using DINOv2 - ResearchGate, accessed on April 19, 2026, https://www.researchgate.net/publication/394827667_Weed_Classification_with_Drone_Image_Using_DINOv2
17. DINOv2.cpp: Accelerating Edge Inference with C++ and GGML | Projects - Alex Lavaee, accessed on April 19, 2026, <https://alexlavaee.me/projects/dinov2cpp/>
18. [R] Reducing DINOv2 FLOPs by 40% and improving performance - Reddit, accessed on April 19, 2026, https://www.reddit.com/r/MachineLearning/comments/1ku6rll/r_reducing_dinov2_flops_by_40_and_improving/
19. Real-Time Pest Detection and Identification System Using Deep Learning - ijarset, accessed on April 19, 2026, <https://ijarset.co.in/Paper31902.pdf>
20. Impatient Queuing for Intelligent Task Offloading in Multi-Access Edge Computing - NEC Laboratories Europe, accessed on April 19, 2026, https://neclab.eu/fileadmin/user_upload/Impatient_Queueing_for_Intelligent_Task_Offloading_in_Multi-Access_Edge_Computing_pre-print.pdf
21. Impatient Queuing for Intelligent Task Offloading in Multi-Access Edge Computing - arXiv, accessed on April 19, 2026, <https://arxiv.org/pdf/2105.11727>
22. M/M/1/K Queueing Model | Real Statistics Using Excel, accessed on April 19, 2026, <https://real-statistics.com/probability-functions/queueing-theory/m-m-1-k-queueing-model/>
23. Analysis of Queueing System MMPP/M/K/K with Delayed Feedback - MDPI, accessed on April 19, 2026, <https://www.mdpi.com/2227-7390/7/11/1128>
24. M/M/1 Queueing System - EventHelix, accessed on April 19, 2026, <https://www.eventhelix.com/congestion-control/m-m-1/>
25. ImanRHT/MM1K_Queue_Simulation: A Performance Analysis of the M/M/1/K Queue Model via Discrete Event Simulation with Varied Service Orders - GitHub, accessed on April 19, 2026, https://github.com/ImanRHT/MM1K_Queue_Simulation
26. M/M/1 Queueing System-Three Examples - YouTube, accessed on April 19, 2026, <https://www.youtube.com/watch?v=bkyNocmLUPU>
27. Queueing model for IIoT task executing by MEC - ResearchGate, accessed on April 19, 2026, https://www.researchgate.net/figure/Queueing-model-for-IIoT-task-executing-by-MEC_fig2_372982218
28. Resource-Constrained Edge AI Solution for Real-Time Pest and Disease Detection in Chili Pepper Fields - MDPI, accessed on April 19, 2026, <https://www.mdpi.com/2077-0472/16/2/223>
29. Raspberry Pi AI Kit: Edge AI Computing for Makers and Developers - Think Robotics,

- accessed on April 19, 2026,
<https://thinkrobotics.com/blogs/product-reviews-buying-guides/raspberry-pi-ai-kit-edge-ai-computing-for-makers-and-developers>
30. Benchmarking Deep Learning Models for Object Detection on Edge Computing Devices - arXiv, accessed on April 19, 2026, <https://arxiv.org/html/2409.16808v1>
 31. Pestopia: Indian Pests and Pesticides Dataset - Kaggle, accessed on April 19, 2026, <https://www.kaggle.com/datasets/shruthisindhura/pestopia>
 32. Pestopia: Indian Pests and Pesticides Dataset - Kaggle, accessed on April 19, 2026, <https://www.kaggle.com/datasets/shruthisindhura/pestopia/code>
 33. Pest Prediction and Pesticides Recommendation Using Deep Learning - Impactfactor, accessed on April 19, 2026, <https://impactfactor.org/PDF/IJDDT/16/IJDDT.Vol16.Issue16s.Article46.pdf>
 34. Pest Dataset V2 - Kaggle, accessed on April 19, 2026, <https://www.kaggle.com/datasets/ibrahimagabardiop/pestaiddatasetv2>
 35. Impact of transgenic events on *Helicoverpa armigera* (Hubner) and *Spodoptera litura* (Fabricius) on cotton in India - ResearchGate, accessed on April 19, 2026, https://www.researchgate.net/publication/283019656_Impact_of_transgenic_events_on_Helicoverpa_armigera_Hubner_and_Spodoptera_litura_Fabricius_on_cotton_in_India
 36. Seasonal abundance of *Helicoverpa armigera* Hübner and *Scirpophaga incertulus* Walker in Tarai of Uttarakhand - FAO AGRIS, accessed on April 19, 2026, <https://agris.fao.org/search/en/providers/122648/records/647356aa53aa8c89630684b9>
 37. PDARS V1 - Kaggle, accessed on April 19, 2026, <https://www.kaggle.com/code/shruthisindhura/pdars-v1/data>
 38. Implementation dino v2 for remote sensing with huggingface transformers - GitHub, accessed on April 19, 2026, <https://github.com/chagmgang/dinov2-remote-sensing>
 39. PiDog Rpi5 vs Rpi4 using ChatGPT - Robotic kit for Raspberry Pi - Sunfounder Forum, accessed on April 19, 2026, <https://forum.sunfounder.com/t/pidog-rpi5-vs-rpi4-using-chatgpt/3824>
 40. Real Time Inference on Raspberry Pi 4 and 5 (40 fps!) - PyTorch documentation, accessed on April 19, 2026, https://docs.pytorch.org/tutorials/intermediate/realtime_rpi.html
 41. Pi 4 vs Pi 5 NAS Battle: 60x Faster? Shocking Benchmark Results - YouTube, accessed on April 19, 2026, <https://www.youtube.com/watch?v=hmcbGL78V9k>
 42. Introducing the Raspberry Pi AI HAT+ 2: Generative AI on Raspberry Pi 5, accessed on April 19, 2026, <https://www.raspberrypi.com/news/introducing-the-raspberry-pi-ai-hat-plus-2-generative-ai-on-raspberry-pi-5/>
 43. Edge AI In Smart Agriculture - Meegle, accessed on April 19, 2026, https://www.meegle.com/en_us/topics/edge-ai-solutions/edge-ai-in-smart-agriculture
 44. Deploying Real-Time Object Detection and Obstacle Avoidance For Smart Mobility Using Edge-Ai - TechRxiv, accessed on April 19, 2026, https://www.techrxiv.org/users/910501/articles/1284384/master/file/data/Edge_AI_Object_Detection_Smart_Mobility_Tyagi_v1/Edge_AI_Object_Detection_Smart_Mobility_Tyagi_v1.pdf
 45. Accelerating Vision AI Inference with TensorRT: YOLOv8 and DINOv2 Optimization in Practice | by Thuan Bui Huy | Medium, accessed on April 19, 2026,

- <https://medium.com/@testth02/accelerating-vision-ai-inference-with-tensorrt-yolov8-and-dinov2-optimization-in-practice-287acd4c73e1>
46. Feedback about Real Time Inference on Raspberry Pi 4 and 5 (40 fps!) #3684 - GitHub, accessed on April 19, 2026, <https://github.com/pytorch/tutorials/issues/3684>
 47. DLCPD-25: A Large-Scale and Diverse Dataset for Crop Disease and Pest Recognition, accessed on April 19, 2026, https://pubmed.ncbi.nlm.nih.gov/41305306/?utm_source=FeedFetcher&utm_medium=rss&utm_campaign=None&utm_content=0MUDgoX_3AKO_8FjIHbDSim-inHbHvZmzgZIOj5My8q&fc=None&ff=20251127112313&v=2.18.0.post22+67771e2
 48. Comparative Analysis of Pre-trained Deep Learning Models and DINOv2 for Cushing's Syndrome Diagnosis in Facial Analysis - arXiv, accessed on April 19, 2026, <https://arxiv.org/html/2501.12023v1>
 49. Transfer Learning-Based Multi-Class Pest Detection using a Real-Time Object Detection Framework with Control-Oriented Grouping for Smart Agriculture - ResearchGate, accessed on April 19, 2026, https://www.researchgate.net/publication/403067947_Transfer_Learning-Based_Multi-Class_Pest_Detection_using_a_Real-Time_Object_Detection_Framework_with_Control-Oriented_Grouping_for_Smart_Agriculture
 50. Precision Corn Pest Detection: Two-Step Transfer Learning for Beetles (Coleoptera) with MobileNet-SSD - MDPI, accessed on April 19, 2026, <https://www.mdpi.com/2077-0472/13/12/2287>
 51. Real-Time Pest Detection and Identification System Using Deep Learning - ResearchGate, accessed on April 19, 2026, https://www.researchgate.net/publication/403058179_Real-Time_Pest_Detection_and_Identification_System_Using_Deep_Learning
 52. Optimal Security Task Offloading in Cognitive IoT Networks: Provably Optimal Threshold Policies and Model-Free Learning - MDPI, accessed on April 19, 2026, <https://www.mdpi.com/2624-831X/7/2/30>
 53. AgriSecure: A Fog Computing-Based Security Framework for Agriculture 4.0 via Blockchain, accessed on April 19, 2026, <https://www.mdpi.com/2227-9717/11/3/757>
 54. (PDF) A Fog-Based Security Framework for Smart Agriculture - ResearchGate, accessed on April 19, 2026, https://www.researchgate.net/publication/392087993_A_Fog-Based_Security_Framework_for_Smart_Agriculture
 55. Cyber Security Intrusion Detection for Agriculture 4.0: Machine Learning-Based Solutions, Datasets, and Future Directions - IEEE/CAA Journal of Automatica Sinica, accessed on April 19, 2026, <https://www.ieee-jas.net/article/doi/10.1109/JAS.2021.1004344>
 56. Transforming Agriculture with High Performance Computing: Sustainable Smart Farming – An Experimental Study | Premier Science, accessed on April 19, 2026, <https://premierscience.com/wp-content/uploads/2026/01/5-pjs-25-1214.pdf>
 57. Emergent Intrusion Detection System for Fog Enabled Smart Agriculture Using Federated Learning and Blockchain Technology - Brunel University Research Archive, accessed on April 19, 2026, <https://bura.brunel.ac.uk/bitstream/2438/31556/1/FullText.pdf>
 58. Reinforcement Learning-Driven Task Offloading and Resource Allocation in Wireless IoT

- Networks - IEEE Xplore, accessed on April 19, 2026,
<https://ieeexplore.ieee.org/iel8/6287639/10820123/10971941.pdf>
59. Queue-Aware Multi-Objective Edge Offloading and Resource Provisioning for Dense IoT Networks Under Synthetic Workload Dynamics - ResearchGate, accessed on April 19, 2026,
https://www.researchgate.net/publication/403040758_Queue-Aware_Multi-Objective_Edge_Offloading_and_Resource_Provisioning_for_Dense_IoT_Networks_Under_Synthetic_Workload_Dynamics
 60. Multi-Agent DRL for Queue-Aware Task Offloading in Hierarchical MEC-Enabled Air-Ground Networks - arXiv, accessed on April 19, 2026,
<https://arxiv.org/html/2503.03391v1>
 61. (PDF) Task Offloading Optimization Using PSO in Fog Computing for the Internet of Drones, accessed on April 19, 2026,
https://www.researchgate.net/publication/387522950_Task_Offloading_Optimization_Using_PSO_in_Fog_Computing_for_the_Internet_of_Drones
 62. Optimized AI-IoT Solution for Real-Time Pest Identification in Smart Agriculture - Bright Publisher, accessed on April 19, 2026,
<https://bright-journal.org/Journal/index.php/JADS/article/download/810/546>
 63. Delay-aware partial task offloading using multicriteria decision model in IoT, access